🧠🔥 **REAL DEVOPS INTERVIEW Q&A — A TO Z PRODUCTION TROUBLESHOOTING**
🚀 **Deep Practical Version — Commands + Check + Fix + Verify + Real Examples**

🔥 **1. WHAT WILL YOU DO IF DEPLOYMENT FAILS?**
👉 Production deployment fail ayithe first panic avvakunda **exactly which stage failed** ani identify chestanu.
👉 Deployment pipeline usually:
Code → Build → Test → Docker Build → Push Image → Deploy → Health Check
👉 First Jenkins/GitHub Actions pipeline open chesi failed stage identify chestanu.
📌 **Meaning:** Entire pipeline ni blindly rerun cheyyakunda failure point ni first isolate cheyyali.

🔍 **Ela Check Cheyyali?**
kubectl get deployments
kubectl get pods
kubectl get events --sort-by=.lastTimestamp
👉 Jenkins/GitHub Actions lo failed stage logs kuda check chestanu.
👉 Kubernetes deployment status:
kubectl rollout status deployment/myapp
👉 Previous rollout history:
kubectl rollout history deployment/myapp

💉 **Example**
👉 Docker build stage fail ayindi anuko.
Log:
npm ERR! package not found
👉 App dependency issue ani identify chestanu.
👉 Dockerfile / package configuration check chestanu.

🔧 **Ela Fix Cheyyali?**
👉 Missing dependency add chestanu.
👉 Docker image rebuild chestanu.
docker build -t myapp:v2 .
👉 Registry ki push chestanu.
docker push <registry>/myapp:v2
👉 Deployment update chestanu.
kubectl set image deployment/myapp myapp=<registry>/myapp:v2

🚨 **Production already broken ayithe?**
👉 Immediate recovery important.
kubectl rollout undo deployment/myapp
👉 Previous stable version ki rollback chestanu.

✅ **Ela Verify Cheyyali?**
kubectl rollout status deployment/myapp
kubectl get pods
kubectl get deployment myapp
👉 Pods Running / Ready lo undali.
📌 **Real production idea:**
First restore service → then root cause → then permanent fix.

🚨 **2. WHAT WILL YOU DO IF KUBERNETES POD IS CRASHING?**
👉 Pod crash ayithe first pod status check chestanu.
🔍 **Ela Check Cheyyali?**
kubectl get pods
👉 Example:
NAME READY STATUS RESTARTS

payment-api-7d9c8 0/1 CrashLoopBackOff 8
👉 Detailed information:
kubectl describe pod payment-api-7d9c8
👉 Application logs:
kubectl logs payment-api-7d9c8
👉 Previous crashed container logs:
kubectl logs payment-api-7d9c8 --previous

🧠 **Important**
👉 CrashLoopBackOff itself root cause kaadu.
👉 Kubernetes repeatedly container start cheyyadaniki try chestundi, container immediately crash avtundi.

🔎 **Common Reasons**
👉 Wrong environment variable
👉 Wrong database URL
👉 Application code error
👉 Missing secret
👉 Out of memory
👉 Wrong container command
👉 Dependency unavailable

🍼 **Example**
Error: DATABASE_URL is undefined
👉 Environment variable missing ani identify chestanu.

🔧 **Ela Fix Cheyyali?**
kubectl get deployment payment-api -o yaml
👉 Environment variables check chestanu.
Secret check:
kubectl get secrets
ConfigMap:
kubectl get configmaps
👉 Correct configuration apply chesi rollout restart:
kubectl rollout restart deployment/payment-api

✅ **Verify**
kubectl get pods
kubectl logs <pod-name>
👉 Running and READY 1/1 ravali.

📈 **3. WHAT WILL YOU DO IF HIGH TRAFFIC COMES SUDDENLY?**
👉 First application actually overloaded ayinda or external issue aa ani check chestanu.
🔍 **Ela Check Cheyyali?**
kubectl top pods
kubectl top nodes
👉 HPA status:
kubectl get hpa
👉 Detailed:
kubectl describe hpa myapp
👉 Deployment replicas:
kubectl get deployment myapp

🧠 **Example**
Normal:
Pods = 5

CPU = 45%
Sale traffic:
Pods = 5
CPU = 95%
👉 HPA properly configured unte:
5 → 10 → 20 pods

🔧 **Immediate Fix**
Temporary manual scaling:
kubectl scale deployment myapp --replicas=20
👉 But permanent solution manual scaling kaadu.
👉 HPA configure cheyyali.
Example:
minReplicas: 5
maxReplicas: 50
targetCPUUtilizationPercentage: 70

🔎 **Load Balancer Check**
kubectl get svc
AWS environment lo ALB/NLB health and metrics kuda check chestanu.

✅ **Verify**
kubectl get hpa
kubectl get pods
kubectl top pods
👉 CPU gradually normal range ki ravali.
📌 **Real production idea:**
Traffic spike → Metrics → HPA → More pods → Load distribution → Recovery.

💣 **4. WHAT WILL YOU DO IF PRODUCTION IS COMPLETELY DOWN?**
👉 Production down ayithe first objective **service restore**.
👉 Immediate ga deep investigation lo time waste cheyyanu.

🔍 **Step 1 — Check Application**
kubectl get pods -A
kubectl get deployments -A
kubectl get svc -A

🔍 **Step 2 — Check Recent Deployment**
kubectl rollout history deployment/myapp
👉 Recent deployment immediately before outage ayithe strong suspect.

🔍 **Step 3 — Check Events**
kubectl get events -A --sort-by=.lastTimestamp

🔧 **Quick Recovery**
Recent deployment cause ani confirm ayithe:
kubectl rollout undo deployment/myapp
👉 Then:
kubectl rollout status deployment/myapp

🧪 **Example**
14:00 → v2 deployed
14:05 → Errors increased
14:07 → Production down
👉 v2 suspicious.

👉 Rollback:
kubectl rollout undo deployment/myapp
👉 Service restore.

🧠 **After Recovery**
👉 Logs analyze
👉 Root cause identify
👉 Developer team involve
👉 Permanent fix
👉 RCA document
📌 **Production rule:**
Recover first → Investigate deeply → Prevent recurrence.

🔐 **5. WHAT WILL YOU DO IF YOU SEE A SECURITY ISSUE?**
👉 Security incident ayithe first affected resource identify chestanu.
🔍 **Check**
kubectl get pods -A
kubectl get networkpolicies -A
kubectl get secrets
AWS side:
aws cloudtrail lookup-events
👉 Suspicious IAM activity, configuration changes, API calls investigate chestanu.

🍼 **Example**
👉 Multiple failed login attempts:
10.10.10.10
10.10.10.11
10.10.10.12
👉 Possible brute-force activity.

🔧 **Fix**
👉 WAF rule add cheyyachu.
👉 Rate limiting implement cheyyachu.
👉 Compromised credentials unte immediately rotate/revoke cheyyali.
👉 IAM permissions review cheyyali.
📌 **Important:** Security issue ni simply application restart chesi solve cheyyakudadhu.

✅ **Verify**
👉 Suspicious requests reduce ayyaya?
👉 Unauthorized access stop ayyinda?
👉 CloudTrail lo unexpected activity unda?
👉 Secrets exposed ayithe rotation complete ayyinda?

⚡ **6. WHAT WILL YOU DO IF CI/CD PIPELINE IS VERY SLOW?**
👉 First total pipeline duration break down chestanu.
🔍 **Check**
Build → 10 min
Testing → 5 min
Docker → 8 min
Deploy → 2 min
👉 Docker build or tests bottleneck ayithe identify chestanu.

🔧 **Fix**
👉 Docker layer caching use chestanu.
👉 Dependencies cache chestanu.

👉 Independent tests parallel ga run chestanu.
👉 Unnecessary pipeline steps remove chestanu.
👉 Large Docker image optimize chestanu.

🍼 **Example**
Before:
Pipeline = 25 minutes
After optimization:
Pipeline = 7 minutes
📌 **Goal:** Speed kosam quality checks remove cheyyadam kaadu.

🛑 **7. WHAT WILL YOU DO IF DATABASE IS SLOW?**
👉 Database slow ayithe application kuda slow avtundi.
🔍 **Check RDS**
AWS CloudWatch metrics:
👉 CPU
👉 Database connections
👉 Read/write latency
👉 IOPS
👉 Storage

🔍 **Application Side**
👉 Slow queries identify chestanu.
Example:
SELECT * FROM orders
WHERE customer_id = 12345;
👉 Required index lekapothe full table scan jaragachu.

🔧 **Fix**
👉 Query optimization
👉 Proper indexes
👉 Connection pooling
👉 Read replicas
👉 Caching using Redis

🍼 **Example**
Before:
Query = 5 seconds
After proper index:
Query = 100 ms

✅ **Verify**
👉 DB latency reduce ayyinda?
👉 CPU normal ayyinda?
👉 Application response time improve ayyinda?

🔄 **8. HOW WILL YOU DEPLOY WITHOUT DOWNTIME?**
👉 Rolling update use chestanu.
👉 Old pods immediate ga delete cheyyanu.
👉 New pods first start chestanu.
👉 New pods healthy ayyaka traffic gradually shift chestanu.

🔍 **Check**
kubectl get deployment myapp
kubectl get pods

🔧 **Deployment Strategy**

```
strategy:
 type: RollingUpdate
 rollingUpdate:
  maxUnavailable: 0
  maxSurge: 1
```

👉 Readiness probe configure chestanu.

Example:

```
readinessProbe:
 httpGet:
  path: /health
  port: 8080
```

🚀 **Deploy**

```
kubectl apply -f deployment.yaml
```

👉 Watch:

```
kubectl rollout status deployment/myapp
```

🚨 **Rollback**

```
kubectl rollout undo deployment/myapp
```

📌 **Real idea:** New version healthy ga prove ayye varaku old healthy pods service ni support chestayi.

💰 **9. WHAT WILL YOU DO IF CLOUD COST IS VERY HIGH?**

👉 First blindly resources delete cheyyanu.

🔍 **Check**

AWS billing/cost reports check chestanu.

👉 EC2 utilization
👉 EBS volumes
👉 NAT Gateway
👉 Load Balancers
👉 RDS
👉 EKS nodes
👉 S3

🔎 **Example**

EC2 CPU average = 10%

👉 Instance over-provisioned ayyi undachu.

🔧 **Fix**

👉 Right-size instances
👉 Remove unused EBS
👉 Scale workloads according to demand
👉 Use Savings Plans/Reserved capacity where appropriate
👉 EKS node utilization optimize cheyyadam
👉 S3 lifecycle policies use cheyyadam

🧠 **Important**

👉 Cost optimization ante production resources blindly reduce cheyyadam kaadu.

📌 **Performance + Availability + Security maintain chestu cost reduce cheyyali.**

🧩 **10. HOW WILL YOU DEBUG IF THERE ARE NO LOGS?**

👉 Logs lekunda root cause identify cheyyadam difficult.

🔍 **First Check**

```
kubectl logs <pod-name>
```

👉 Logs empty ayithe application logging configuration check chestanu.

🔧 **Fix**
👉 Application stdout/stderr logging enable chestanu.
👉 Centralized logging setup chestanu.
Examples:
Fluent Bit
CloudWatch
ELK
OpenSearch

🧠 **Kubernetes Important**
👉 Container logs:
kubectl logs <pod>
👉 Multiple replicas unte each pod logs separate ga untayi.
👉 Production lo centralized logging important.

✅ **Verify**
👉 New request generate chestanu.
👉 Logs centralized system lo appear avtunnaya check chestanu.

🔴 **11. WHAT WILL YOU DO IF POD IS IN PENDING STATE?**
👉 Pending ante pod successfully schedule/start avvaledu.
🔍 **Check**
kubectl get pods
kubectl describe pod <pod-name>
👉 Events section especially check chestanu.

🧠 **Common Reasons**
👉 Insufficient CPU
👉 Insufficient memory
👉 Node selector mismatch
👉 Taints/tolerations problem
👉 PVC unavailable

💉 **Example**
0/3 nodes are available:
Insufficient cpu
👉 Nodes daggara required CPU ledu.

🔧 **Fix**
👉 Node capacity increase
👉 Additional node add
👉 Resource requests review
👉 Cluster autoscaler verify

✅ **Verify**
kubectl get pods
👉 Pod:
Pending → ContainerCreating → Running

🔴 **12. WHAT WILL YOU DO IF POD IS IN OOMKILLED?**
👉 OOMKilled ante container available memory limit exceed chesindi.
🔍 **Check**
kubectl describe pod <pod-name>

👉 Resource configuration:
kubectl get deployment myapp -o yaml

🍼 **Example**
resources:
 limits:
  memory: "256Mi"
👉 Application actual requirement:
400Mi
👉 Container memory limit exceed → OOMKilled.

🔧 **Fix**
👉 First memory leak unda check cheyyali.
👉 Application optimize cheyyali.
👉 Correct memory request/limit configure cheyyali.
Example:
resources:
 requests:
  memory: "256Mi"
 limits:
  memory: "512Mi"

⚠️ **Important**
👉 Just memory limit increase cheyyadam permanent fix kaadu.
📌 **Memory leak unte application code fix cheyyali.**

🌐 **13. WHAT WILL YOU DO IF SERVICE IS NOT REACHABLE?**
🔍 **Check**
kubectl get svc
kubectl describe svc myapp
kubectl get endpoints myapp
👉 Service ki backend pods select avtunnaya check chestanu.

🍼 **Example**
Service selector:
selector:
 app: backend
Pod label:
app: frontend
👉 Match avvadu.
👉 Service backend pods ni find cheyyaledu.

🔧 **Fix**
👉 Correct labels/selectors configure chestanu.

✅ **Verify**
kubectl get endpoints myapp
👉 Endpoints IPs appear avvali.

⚖️ **14. WHAT WILL YOU DO IF LOAD BALANCER IS RETURNING 503?**
👉 503 usually backend available/healthy ga lekapovadam indicate cheyyachu.
🔍 **Check**
kubectl get pods
kubectl get svc
kubectl describe svc myapp
AWS ALB use chestunte:

👉 Target health
👉 Listener
👉 Security groups
👉 Target group
👉 Health check path
check chestanu.

💉 **Example**
ALB health check:
/health
Application actually:
/status
👉 ALB targets unhealthy.

🔧 **Fix**
👉 Correct health-check path configure chestanu.

✅ **Verify**
👉 Targets Healthy
👉 ALB 503 errors reduce
👉 Application request successful

🧱 **15. WHAT WILL YOU DO IF DOCKER IMAGE BUILD FAILS?**
🔍 **Check**
docker build -t myapp:v1 .
👉 Error output carefully read chestanu.

💉 **Example**
COPY failed: file not found
👉 Dockerfile wrong path.

🔧 **Fix**
👉 Dockerfile paths verify chestanu.
👉 .dockerignore kuda check chestanu.
👉 Dependencies correct ga install chestunnaya check chestanu.

✅ **Verify**
docker build -t myapp:v1 .
docker run -p 8080:8080 myapp:v1
👉 Local container healthy ga run ayithe registry ki push chestanu.

☁️ **16. WHAT WILL YOU DO IF KUBERNETES CANNOT PULL IMAGE?**
🔍 **Check**
kubectl describe pod <pod-name>
👉 Event:
ImagePullBackOff
ErrImagePull

🔎 **Common Reasons**
👉 Wrong image name
👉 Wrong tag
👉 Private registry authentication missing
👉 Image doesn't exist
👉 ECR permissions missing

💉 **Example**
Deployment:
myapp:v10
Registry lo:
myapp:v9
👉 v10 image exist kaadu.

🔧 **Fix**
Correct image:
kubectl set image deployment/myapp myapp=<registry>/myapp:v9
👉 Private ECR ayithe node/pod ki correct IAM permissions verify chestanu.

✅ **Verify**
kubectl get pods
👉 ImagePullBackOff disappear ayi Running ravali.

🔐 **17. WHAT WILL YOU DO IF POD GETS "SECRET NOT FOUND"?**
🔍 **Check**
kubectl get secrets
kubectl describe pod <pod-name>
👉 Deployment YAML:
kubectl get deployment myapp -o yaml

💉 **Example**
Deployment expects:
payment-secret
Cluster lo:
payments-secret
👉 Name mismatch.

🔧 **Fix**
👉 Correct Secret create/reference cheyyali.
kubectl get secret payment-secret
👉 Deployment reference correct cheyyali.

✅ **Verify**
kubectl rollout restart deployment/myapp
kubectl get pods

💾 **18. WHAT WILL YOU DO IF EBS DISK IS FULL?**
👉 EC2 server lo disk full ayithe application fail avvachu.
🔍 **Check**
df -h
👉 Large directories:
du -sh /*
👉 Logs:
du -sh /var/log/*

💉 **Example**
/dev/xvda1 = 100%
👉 Disk completely full.

🔧 **Fix**
👉 Unnecessary files/logs clean cheyyali.
👉 Log rotation configure cheyyali.

👉 EBS volume expand cheyyali if required.

✅ **Verify**
df -h
👉 Disk utilization safe range lo undali.

🌐 **19. WHAT WILL YOU DO IF DNS IS NOT WORKING?**
🔍 **Check**
nslookup example.com
or:
dig example.com
👉 DNS record correct IP/load balancer ki point chestunda check chestanu.

🍼 **Example**
www.example.com
↓
Old Load Balancer
👉 New ALB create chesaru but DNS old ALB ki point chestundi.

🔧 **Fix**
👉 Route 53 record correct target ki update chestanu.

✅ **Verify**
nslookup example.com
👉 Correct endpoint resolve avvali.

🔥 **20. WHAT WILL YOU DO IF APPLICATION RESPONSE IS VERY SLOW?**
👉 First application slow aa, database slow aa, network slow aa isolate chestanu.
🔍 **Check**
kubectl top pods
kubectl top nodes
👉 Application logs
👉 Load balancer latency
👉 Database latency
👉 External API latency
check chestanu.

🍼 **Example**
Frontend → 100ms
Backend → 150ms
Database query → 4 seconds
👉 Main bottleneck database.

🔧 **Fix**
👉 Slow query optimize
👉 Index
👉 Connection pool
👉 Cache
👉 DB scaling

✅ **Verify**
Database = 4 sec → 200 ms
API = 5 sec → 300 ms
📌 **Real DevOps thinking:** symptom ni fix cheyyadam kaadu; bottleneck ni identify cheyyali.

🔄 **21. WHAT WILL YOU DO IF ROLLOUT IS STUCK?**

🔍 **Check**

kubectl rollout status deployment/myapp

👉 Pods:

kubectl get pods

👉 Events:

kubectl get events --sort-by=.lastTimestamp

🍼 **Example**

New pods → 0/1 Ready

👉 Readiness probe failing.

🔧 **Fix**

👉 Application health endpoint check chestanu.

kubectl describe pod <pod-name>

kubectl logs <pod-name>

👉 Correct readiness configuration.

🚨 **If production impact**

kubectl rollout undo deployment/myapp

✅ **Verify**

kubectl rollout status deployment/myapp

🧠 **22. WHAT WILL YOU DO IF NODE GOES DOWN?**

👉 First node status check chestanu.

kubectl get nodes

Example:

node-1 NotReady

node-2 Ready

node-3 Ready

👉 Node issue investigate chestanu.

kubectl describe node node-1

🔧 **Fix**

👉 EKS environment lo unhealthy node replace cheyyachu.

👉 Cluster Autoscaler/Karpenter configured unte replacement automatic ga jaragachu.

👉 Critical workloads other healthy nodes lo schedule avvali.

✅ **Verify**

kubectl get nodes

kubectl get pods -o wide

👉 Critical pods healthy nodes lo run avvali.

📊 **23. WHAT WILL YOU DO IF MONITORING ALERTS ARE NOT COMING?**

🔍 **Check**

👉 CloudWatch metrics

👉 Prometheus targets

👉 Alertmanager

👉 Notification channel

Prometheus example

kubectl get pods -n monitoring

👉 Monitoring pods healthy ga unnaya check chestanu.

🔧 **Fix**
👉 Scrape configuration
👉 Alert rule
👉 Alertmanager configuration
👉 SNS/Slack/PagerDuty integration
check/fix chestanu.

🧠 **Why Important?**
👉 Monitoring itself down ayithe application healthy unna kuda DevOps ki problem teliyadu.
📌 **Observability failure is also a production risk.**

🚨 **24. WHAT WILL YOU DO IF DATABASE CONNECTIONS ARE EXHAUSTED?**
🔍 **Check**
👉 RDS connections metric
👉 Application connection pool
👉 DB configuration

🧪 **Example**
Max connections = 100
Active = 100
👉 New requests database connect avvalevu.

🔧 **Fix**
👉 Connection pool tuning
👉 Idle connections cleanup
👉 Application connection leak fix
👉 DB capacity review
👉 Read traffic replicas/cache ki shift cheyyadam

✅ **Verify**
👉 Active connections stable ga undali.
👉 Application errors reduce avvali.

🔥 **25. WHAT WILL YOU DO IF S3 ACCESS DENIED COMES?**
🔍 **Check**
👉 IAM role
👉 IAM policy
👉 Bucket policy
👉 KMS permissions if encrypted

AWS CLI
aws s3 ls s3://my-bucket

🧪 **Example**
AccessDenied
👉 Application IAM role ki S3 GetObject permission ledu.

🔧 **Fix**
👉 Least-privilege IAM permission add chestanu.
📌 **Admin access ivvadam solution kaadu.**

✅ **Verify**
aws s3 ls s3://my-bucket
👉 Required operation work avvali.

🚀 **26. WHAT WILL YOU DO IF APPLICATION MEMORY LEAKS?**

👉 Memory gradually increase avtundi.

🔍 **Check**

kubectl top pods

👉 Example:

10:00 → 200Mi
11:00 → 400Mi
12:00 → 700Mi
13:00 → OOMKilled

👉 Strong memory leak indication.

🔧 **Fix**

👉 Developer tho heap/profile analysis
👉 Application code fix
👉 Temporary memory increase if required
👉 Restart strategy only temporary workaround.

🧠 **Important**

📌 **Restarting leaking application repeatedly is not permanent solution.**

🧪 **27. WHAT WILL YOU DO IF APPLICATION WORKS LOCALLY BUT NOT IN PRODUCTION?**

👉 Environment difference identify chestanu.

🔍 **Check**

👉 Environment variables
👉 Secrets
👉 Configuration
👉 Network access
👉 Database URL
👉 Dependency versions
👉 Container image

🧪 **Example**

Local:
localhost:5432
Production:
RDS endpoint

👉 Production config wrong ga undachu.

🔧 **Fix**

👉 Environment-specific configuration correct chestanu.
👉 Same tested Docker image use chestanu.

🛡️ **28. WHAT WILL YOU DO IF A DEPLOYMENT INTRODUCES ERRORS?**

👉 First error start ayina exact time identify chestanu.

Before deployment → Error 1%
After deployment → Error 30%

👉 Strong deployment correlation.

🔍 **Check**

kubectl rollout history deployment/myapp
kubectl logs <pod-name>

🔧 **Fix**

Immediate:

kubectl rollout undo deployment/myapp
Permanent:
👉 Developer root cause fix
👉 Test
👉 New image
👉 Controlled redeployment

🧠🔥 **29. WHAT IS YOUR GENERAL TROUBLESHOOTING APPROACH?**
👉 Nenu every production issue ki random commands run cheyyanu.
Step 1 — Understand
👉 Exact symptom enti?
Step 2 — Impact
👉 Enta users affected?
Step 3 — Recent Changes
👉 Recent deployment/change unda?
Step 4 — Metrics
👉 CPU?
👉 Memory?
👉 Latency?
👉 Error rate?
Step 5 — Logs
👉 Application logs
👉 Kubernetes events
👉 Infrastructure logs
Step 6 — Isolate
👉 DNS?
👉 Network?
👉 Load balancer?
👉 Compute?
👉 Database?
👉 Application?
Step 7 — Recover
👉 Rollback / restart / scale / failover depending on cause.
Step 8 — Verify
👉 Service actually recovered aa?
Step 9 — Prevent
👉 Permanent fix
👉 Alert
👉 Automation
👉 Documentation

⏳🔥 **30. REAL DEVOPS INCIDENT THINKING**
🚨 **Production issue vachindi anuko:**
User reports website slow
↓
Check monitoring
↓
Check latency + error rate
↓
Check recent deployment
↓
Check Load Balancer
↓
Check Pods
↓

Check CPU / Memory
↓
Check Database
↓
Find Root Cause
↓
Immediate Recovery
↓
Verify
↓
RCA
↓
Permanent Fix
↓
Monitoring / Automation

🧠🔥 **FINAL INTERVIEW ANSWER — MOST IMPORTANT**

👉 Interviewer **"How do you troubleshoot production issues?"** ani adigithe:

"First I understand the impact and identify the affected service. Then I check monitoring metrics, logs, recent deployments and infrastructure health. I isolate the problem layer by layer — DNS, load balancer, compute, Kubernetes, application or database. My first priority is to restore service safely, so if a recent deployment caused the issue I may rollback to the last stable version. After recovery, I identify the root cause, implement a permanent fix, verify the system and document the incident through RCA. Finally, I add monitoring or automation to prevent the same issue from happening again."

🔥 **FINAL DEVOPS MASTER FORMULA**

👉 **Don't memorize commands only.**

Real DevOps engineer thinking:

Alert → Understand → Check → Isolate → Diagnose → Fix → Recover → Verify → Prevent

📌 **Commands are tools. Thinking is the actual DevOps skill.**

🧠 **Quick Command Memory**

kubectl get pods
kubectl describe pod <pod>
kubectl logs <pod>
kubectl logs <pod> --previous
kubectl get events --sort-by=.lastTimestamp
kubectl get nodes
kubectl top pods
kubectl top nodes
kubectl get svc
kubectl get endpoints
kubectl get hpa
kubectl rollout status deployment/<name>
kubectl rollout history deployment/<name>
kubectl rollout undo deployment/<name>
kubectl scale deployment <name> --replicas=10
kubectl get deployment -o yaml

👉 Ee commands ni **blind ga run cheyyadam kaadu** — output lo em vachindo chusi next command decide cheyyadam important.

Production troubleshooting = "Command run cheyyadam" kaadu; "output ni interpret chesi next action decide cheyyadam."